GUIA DE ESTUDO

Cursor sem segredos

Básico, intermediário e avançado. Para quem programa em arquivo de texto, guarda o trabalho no pendrive e tem inglês básico.

Guia independente, escrito a partir da documentação pública do Cursor em outubro de 2026. Não é um manual oficial.

COMEÇO

Como ler este livro

Três níveis, do primeiro clique até o agente que trabalha sozinho. Sem vídeo e sem enrolação.

Este livro ensina a usar o Cursor de verdade: a tela, os atalhos, o que pedir, o que revisar e quando deixar o agente trabalhar longe da sua máquina. Cada capítulo termina com um gesto concreto, não com uma lista de nomes de botão.

> **Comece por aqui**
>
> Se você programa em arquivo de texto e o inglês da tela atrapalha, leia Por que o Cursor funciona assim. Os cliques estão em No seu computador, não na nuvem. O dicionário das palavras inglesas está em A tela está em inglês.

Nível 1**Básico**

Por que a ferramenta funciona assim, como fazer a conversa nascer no seu computador e como revisar a mudança.

Nível 2**Intermediário**

Modos, pedidos melhores, regras do projeto, modelos, Git, skills e ferramentas externas.

Nível 3**Avançado**

Agente na nuvem, terminal, subagentes, hooks, revisão de pull request e um jeito de trabalhar que não desanda.

Como os atalhos aparecem

A documentação do Cursor escreve os atalhos no estilo do Mac. No Windows e no Linux, `Cmd` vira Ctrl e `Option` vira Alt. O botão no topo do livro troca todos os atalhos de uma vez. `Shift`, `Tab`, `Enter` e `Esc` continuam iguais.

Se um atalho não fizer o que o livro diz, alguém já o remapeou. Abra a paleta com `Cmd+Shift+P`, procure *Keyboard Shortcuts*, ou use a sequência `Cmd+R` e depois `Cmd+S`.

O que este livro não é

Não é o manual oficial da Cursor e não lista preço de modelo. Preço, nome de modelo e detalhe de tela mudam. O mapa estável está em cursor.com/docs. O que não muda é o ofício: você descreve o resultado, o agente mexe no código, e você confere antes de ficar com a mudança.

> **Como estudar**
>
> Leia um capítulo e faça o exercício na hora, num projeto pequeno seu. Marque o capítulo como lido. O navegador guarda isso só nesta máquina.

Índice

BÁSICO

Por que o Cursor funciona assim

Para quem programa em arquivo de texto, com inglês básico, e quer entender a ferramenta antes de assinar.

Dá para programar a vida inteira num editor de texto e no papel. O Cursor não pede outra profissão. Ele pede que você saiba onde o trabalho está acontecendo e que você leia a alteração antes de ficar com ela. O resto é o ofício que você já tem: dizer o que o programa deve fazer.

Por que a janela parece outra coisa

Um programa continua sendo um arquivo de texto. A janela em volta é uma mesa com gavetas: a lista de arquivos, a busca, o lugar onde o programa roda e o histórico do que mudou. Isso se chama IDE, ambiente de desenvolvimento. O nome assusta. A função não. É o bloco de notas com a pasta do projeto aberta ao lado.

O Cursor é essa mesa mais uma pessoa que escreve no arquivo quando você pede. Você continua dono do texto. A pessoa devolve uma proposta. Você aceita ou joga fora.

Por que a tela está em inglês

O produto é feito em inglês e os botões não foram traduzidos. Inglês básico alcança o nome do botão. Não alcança a decisão escondida atrás dele. Por isso o grupo perde hora: cada um finge que entendeu *Cloud* ou *Last Used*, e a conversa vai para o computador errado. A tradução do que fazer está neste livro, no capítulo A tela está em inglês. O pedido ao agente pode ser em português.

Por que existem dois computadores

Há o seu computador, com os seus arquivos e o pendrive na mesa. Há outro computador, na nuvem, que recebe uma cópia do projeto. Esse segundo computador trabalha com a sua máquina desligada e devolve um pedido de junção, o pull request.

O pendrive não aparece lá porque não está ligado naquele computador. Não é falha do cabo. É outro endereço. O desktop que o agente da nuvem abre é a tela desse segundo computador, não a sua área de trabalho.

Os dois servem. O seu computador serve para o arquivo que está na sua frente. A nuvem serve para um trabalho que pode terminar sem você. Começar na nuvem sem querer é o que faz a gente desistir: a ferramenta parece não ver o que está na mesa.

Por que mudar a configuração não muda esta conversa

Uma conversa nasce num dos dois computadores e fica nele. **Default Environment** (ambiente padrão) só diz onde nasce a próxima. **Last Used** (o último usado) repete o lugar da anterior. Se a anterior foi na nuvem, a próxima nasce na nuvem de novo. Fechar o programa não carrega a conversa de um computador para o outro. É o mesmo caso de trocar a impressora padrão: a folha que já está saindo continua na impressora antiga.

Para trabalhar nos seus arquivos, abra a pasta, deixe o padrão em **Local** (no seu computador) e comece uma conversa nova. Os cliques estão em No seu computador, não na nuvem.

Por que se abre a pasta, e não um arquivo

Um arquivo aberto é uma folha. A pasta é o maço. O agente precisa do maço para achar a folha certa. **Open File** entrega uma folha. **Open Folder** entrega o maço. Sem o maço, o programa muitas vezes só oferece a nuvem, porque não há pasta local para trabalhar.

Por que se lê a diferença antes de aceitar

O agente escreve com segurança mesmo quando erra. A resposta em texto diz a intenção. O **diff** é a folha riscada: o que saiu e o que entrou. Aceitar é rubricar. Sem a rubrica, entra no projeto uma alteração que ninguém leu. Isso vale no papel, no arquivo de texto e aqui.

Por que há três jeitos de escrever

- **Tab** completa a linha que você já está digitando. É um colega olhando por cima do ombro. Não conversa e não segue regra do projeto.
- **Edição inline**, o `Cmd+K`, muda o trecho que você marcou. Serve para uma folha, um pedido curto.
- **Agent** é o pedido de um trabalho: explicar, alterar várias folhas, rodar o programa. Devolve a folha riscada para você decidir.

O que basta para começar

Assinar vale a pena quando estas três coisas estão claras: a conversa está no seu computador, o pedido está em português e diz o resultado, e você lê a alteração antes de aceitar. Quem desiste em geral não falhou em programar. Falhou em descobrir em qual computador a conversa estava. Esse descobrir não precisa ser uma reunião. Está escrito aqui.

> **Para o grupo**
>
> Na primeira hora, uma pessoa compartilha a tela. Antes de qualquer pedido, todos leem o seletor embaixo do campo. Se estiver Cloud, a hora ainda não começou. O roteiro inteiro está em A tela está em inglês.

BÁSICO

O que o Cursor faz

Um editor de código com um agente que lê o projeto, edita arquivos e roda comandos. Três ferramentas cobrem quase todo o dia.

O Cursor é um editor de código com inteligência artificial por dentro. Em volta do texto ficam a lista de arquivos, a busca, o terminal e o histórico do que mudou. Se você vem do bloco de notas, leia antes Por que o Cursor funciona assim. A diferença prática é o agente, que propõe alteração, e a sugestão que completa a linha enquanto você escreve.

As três ferramentas do dia a dia

Ferramenta	Quando usar	O que ela não faz
Tab	Você está digitando e quer a próxima linha, ou as próximas.	Não segue uma regra do projeto. Não conversa.
Edição inline	Uma mudança local, no trecho que você selecionou.	Não aplica as regras de usuário do chat. Não é o lugar de uma refatoração grande.
Agent	Explicar o projeto, alterar vários arquivos, rodar testes, investigar um bug.	Não substitui a sua revisão. Ele erra com confiança.

O Agent junta três peças: as instruções (o que você escreveu, mais as regras do projeto), as ferramentas (buscar, ler, editar, terminal, navegador) e o modelo que você escolheu. O Cursor ajusta esse conjunto para cada modelo. Você não precisa reaprender o editor quando troca o modelo.

O que continua sendo sua função

- Dizer qual resultado você quer, com um jeito de saber se deu certo.
- Apontar o arquivo quando você já sabe qual é. Deixar o agente procurar quando não sabe.
- Ler o diff. Aceitar, rejeitar ou pedir outro caminho.
- Não colar senha, token ou dado de cliente no chat.

> **Uma frase para guardar**
>
> Tab completa o que você já está fazendo. A edição inline muda um trecho. O Agent faz um trabalho e espera a sua revisão.

BÁSICO

No seu computador, não na nuvem

O aplicativo abre no desktop, mas a conversa pode rodar na nuvem. A nuvem não vê o seu pendrive nem os seus arquivos.

Este é o capítulo que evita a perda de tempo. O Cursor no seu computador e o agente na nuvem são coisas diferentes. Dá para abrir o aplicativo aí na mesa e, mesmo assim, a conversa estar numa máquina virtual longe de você.

Os dois lugares

Onde a conversa roda	O que ela enxerga
No seu computador	A pasta que você abriu, o pendrive montado, os arquivos dessa máquina.
Na nuvem	Uma cópia do repositório numa máquina virtual. O desktop dessa máquina não é o seu. O endereço `127.0.0.1` de lá não abre no seu navegador.

A nuvem serve quando você quer que o agente trabalhe sozinho e devolva um pull request. Não serve para ler um pendrive plugado no PC.

Por que a sua mudança volta para a nuvem

Em **Settings** → **Agents** → **Default Environment** o padrão costuma ser **Last Used**. Uma conversa na nuvem faz a próxima nascer na nuvem de novo. Trocar uma conversa no meio do caminho não grava o padrão. Fechar e abrir o Cursor também não move a conversa que já está na nuvem.

Há outro atalho para a nuvem: começar sem pasta aberta. Sem uma pasta local, o seletor só mostra **Cloud**.

Faça isto uma vez

1. No menu **File** (Arquivo), clique em **Open Folder** (Abrir pasta) e escolha a pasta do projeto. **Open File** abre um arquivo só e não basta.
2. Abra **Settings** (Configurações), entre em **Agents** (Agentes) e em **Default Environment** (Ambiente padrão). Escolha **Local** (no seu computador). Não deixe **Last Used** (o último usado): se a última conversa foi na nuvem, a próxima nasce na nuvem.
3. Feche o Cursor e abra de novo. A conversa antiga não muda de lugar.
4. Comece uma conversa nova. Na tela isso aparece como **New Chat** ou **New Agent**.
5. Antes de escrever, olhe o seletor embaixo do campo. Tem de estar no seu computador. Se estiver **Cloud** (nuvem), não envie.

> **Como saber sem perguntar ao agente**
>
> O seletor embaixo do campo é a resposta, antes do envio. Se a conversa já foi para a nuvem, o menu dela tem **Move to** (Mover para) e depois **Local**. Isso só funciona quando a mesma pasta já está aberta no computador. As outras palavras da tela estão em A tela está em inglês.

Pendrive

O agente só lê um pendrive que o sistema operacional já montou na mesma máquina em que ele roda. No Linux, o caminho aparece em `/media` ou `/mnt`. No Windows, como um disco, por exemplo `E:`. Um pendrive no seu PC não atravessa até a nuvem. Não há botão no desktop da nuvem que traga esse USB.

O pendrive continua sendo um jeito inteiro de trabalhar. O GitHub é outro lugar, na internet, e pede conta e conexão. O porquê dessa diferença, e como abrir a pasta do pendrive no Cursor sem essa conta, está em Pendrive e GitHub.

> **Pratique agora**
>
> Abra uma conversa nova com a pasta do projeto aberta. Antes de enviar qualquer pedido, leia o seletor. Se estiver em Cloud, não envie. Corrija o padrão para Local e abra outra conversa.

BÁSICO

A tela está em inglês

O que cada palavra do Cursor significa em português, e um roteiro para o grupo parar de descobrir isso no escuro.

O Cursor não tem a tela em português. Quem finge que leu o menu trava o grupo inteiro. Este capítulo é o dicionário: a palavra que aparece na tela, o que ela quer dizer, e o que fazer com ela. Quando alguém travar numa palavra, a resposta está aqui, não numa reunião para adivinhar.

Onde a conversa roda

Na tela	Em português	O que fazer
File	Arquivo	Menu do canto. É por aqui que se abre a pasta.
Open Folder	Abrir pasta	Escolha a pasta do projeto no seu computador. Sem isso, a conversa cai na nuvem.
Open File	Abrir arquivo	Abre um arquivo só. Não substitui abrir a pasta.
Settings	Configurações	A engrenagem, ou `Cmd+,`.
Cursor Settings	Configurações do Cursor	`Cmd+Shift+J`. Modelos, agentes e privacidade ficam aqui.
Agents	Agentes	A seção em que se escolhe onde a próxima conversa nasce.
Default Environment	Ambiente padrão	O lugar da próxima conversa. Escolha Local.
Local	No seu computador	Vê a pasta aberta e o pendrive montado nessa máquina.
Cloud	Na nuvem	Máquina virtual. Não vê o seu pendrive nem o seu desktop.
Last Used	O último usado	Repete o lugar da conversa anterior. Se a última foi Cloud, a próxima também é. Não deixe este padrão.
New Chat ou New Agent	Conversa nova	A conversa velha não muda de lugar. O teste do padrão Local é sempre uma conversa nova.
Move to	Mover para	No menu da conversa já aberta. Move to → Local tenta trazê-la para o computador. Exige a pasta aberta.
Desktop	Área de trabalho	No agente da nuvem, é a tela da máquina virtual, não a do seu PC.

Pedir, aceitar e recusar

Na tela	Em português	O que fazer
Agent	Agente que faz	Busca, edita arquivos e roda comandos. É o modo de trabalho.
Ask	Perguntar	Só lê e responde. Não altera arquivo. Use para entender antes de autorizar.
Plan	Planejar	Pesquisa, pergunta e escreve o plano. Só edita depois que você aprova.

Na tela	Em português	O que fazer
Debug	Depurar	Investiga um bug que você consegue repetir, com a aplicação rodando.
Design	Desenhar na tela	Você aponta um pedaço da página e o agente mexe no código desse pedaço.
Tab	Tecla Tab	Aceita a sugestão cinza enquanto você digita. Não é uma conversa.
Accept	Aceitar	Fica com a mudança. Leia o diff antes.
Reject	Rejeitar	Descarta a mudança.
diff	Diferença	O que saiu e o que entrou. É a revisão. A frase do agente não substitui o diff.
Models	Modelos	Qual inteligência responde. Mais rápido para o miúdo. Mais capaz para vários arquivos.
Command Palette	Paleta de comandos	`Cmd+Shift+P`. Se o botão não aparecer, procure o nome inglês aqui.
Keyboard Shortcuts	Atalhos de teclado	A lista oficial. Os atalhos deste livro podem ter sido remapeados.

Arquivo, Git e pendrive

Na tela	Em português	O que fazer
Source Control	Controle de versão	O painel do Git. Mostra o que mudou na pasta.
Commit	Gravar uma versão	Você confirma que leu o diff. O agente não precisa commitar no seu lugar.
Branch	Ramo	Uma linha de trabalho separada, para não misturar com o projeto estável.
Push	Enviar	Manda os commits para o repositório remoto.
Pull request	Pedido de junção	Pede para o grupo olhar a branch antes de juntar ao projeto.
Merge	Juntar	Incorpora a branch. Só depois da revisão.
Terminal	Terminal	A tela de comandos. O agente pode rodar teste aqui. Você autoriza o que é perigoso.
Workspace	A pasta aberta	O conjunto de arquivos que o agente local enxerga.
Save to workspace	Salvar na pasta do projeto	Guarda o plano dentro do projeto, para o grupo achar depois.
Indexing	Indexação	O Cursor lendo o projeto para buscar por sentido, não só por palavra.

Regras e permissão para rodar comando

Na tela	Em português	O que fazer
Customize	Personalizar	Onde ficam regras, skills e ferramentas externas.

Na tela	Em português	O que fazer
Rules	Regras	Instruções fixas do projeto. O agente do chat obedece. O Tab não.
Add Rule	Adicionar regra	Cria um arquivo de regra. Também dá para pedir com `/create-rule`.
Approvals & Execution	Aprovações e execução	Em Settings → Agents. Decide se o comando roda sozinho ou pede licença.
Auto-review	Revisão automática	O recomendado. Roda o que é seguro e pergunta o resto.
Allowlist	Lista permitida	Só o que você listou roda sem perguntar.
Run Everything	Rodar tudo	Nada pede licença. Não use no projeto de verdade.
Sign in	Entrar	Entrar na conta. Sem conta, parte do agente não funciona.

Roteiro de uma hora para o grupo

Uma pessoa compartilha a tela. As outras não adivinham o menu: conferem a palavra neste capítulo. A hora acaba com uma mudança pequena aceita, não com uma aula de vocabulário.

1. Abrir a pasta do projeto (Open Folder) e ler o seletor. Se estiver Cloud, parar e corrigir para Local antes de qualquer pedido.
2. Pedir, em português: “Explique este projeto. Aponte por onde eu começo a ler.”
3. Escolher uma mudança pequena. Pedir o resultado, o que não pode mudar e como verificar.
4. Ler o diff juntos. Aceitar (Accept) só o que o grupo entendeu. Rejeitar (Reject) o resto e dizer o que faltou.
5. Anotar a palavra inglesa que travou alguém. Na próxima reunião ela já está neste capítulo. Se não estiver, acrescentem uma linha na regra do projeto pedindo resposta em português.

> **Peça em português**
>
> O menu é em inglês. O pedido não precisa ser. Escreva do jeito que o grupo fala. Se o agente responder em inglês, diga: “Responda em português.” Para isso valer sempre, grave uma regra de usuário em Customize → Rules.

BÁSICO

Pendrive e GitHub

Por que a pasta no pendrive anda fácil e a conexão do GitHub fica parada, e como usar o Cursor sem essa conta.

O pendrive guarda a pasta. Você leva a pasta até o outro computador e ela abre. O GitHub guarda outra cópia, num computador da internet. Para chegar nela, o programa precisa de conta, senha ou chave, internet e permissão. Quando falta uma dessas coisas, a tela fica esperando. Não é o pendrive que falhou. É um caminho diferente, com portaria.

Por que o pendrive é mais simples

Copiar uma pasta não pergunta quem você é. O arquivo sai de um lugar e entra no outro. Dois amigos combinam de quem está com o pendrive e trabalham. Não há servidor no meio.

O limite também é simples. Se os dois editam ao mesmo tempo, cada um na sua cópia, nenhuma das duas sabe o que a outra mudou. Na hora de juntar, sobra folha contra folha. A data do arquivo não resolve: os dois podem ter mudado partes diferentes no mesmo dia.

Por que a conexão do GitHub não completa

GitHub não é a pasta. É um endereço. O verbo da tela costuma ser um destes:

Na tela	Em português	Por que trava
Sign in	Entrar na conta	Sem conta, o endereço não abre a portaria.
Clone	Baixar uma cópia	Traz o projeto da internet para a sua pasta. Pede o endereço certo e permissão.
Push	Enviar daqui para lá	Manda as suas alterações. Sem login, ou sem permissão de escrita, para e espera.
Pull	Trazer de lá para cá	Busca o que o outro já enviou. Sem internet, não há o que buscar.
Authentication failed	A identificação falhou	A senha, o token ou a chave não foi aceita. A pasta do pendrive nem chega a ser lida.
Permission denied	Permissão negada	A conta entrou, mas esse projeto não é seu para escrever.
SSH	Uma chave, em vez de senha	Outro jeito de provar quem você é. Sem a chave instalada, a conexão não passa.

Ficar olhando essa tela não aproxima o pendrive do GitHub. São duas cópias que só se encontram se a portaria abrir. Enquanto a portaria não abre, o trabalho segue na pasta que já está na mão.

Por que a nuvem do Cursor pede essa conexão

O computador da nuvem não tem a sua porta USB. Ele só recebe o projeto se o projeto estiver num endereço da internet, como o GitHub. Por isso o agente da nuvem e o pendrive não se encontram. Para a pasta do pendrive, a conversa tem de ser **Local**, no computador em que o pendrive está montado.

Como trabalhar na pasta do pendrive

1. Espere o sistema mostrar o pendrive. No Linux, em `/media` ou `/mnt`. No Windows, como um disco, por exemplo `E:`.
2. No Cursor, **File → Open Folder** e escolha a pasta dentro do pendrive. Não escolha um arquivo solto.
3. Conversa nova, seletor no seu computador, não em Cloud.
4. Peça em português. Leia o diff. Aceite só o que o grupo entendeu.
5. Antes de tirar o pendrive, ejete pelo sistema. Tirar no meio da gravação corrompe arquivo.
6. Para o amigo, entregue o pendrive ou copie a pasta inteira. Combinem quem edita. Duas edições ao mesmo tempo não se juntam sozinhas.

Por que o Git existe, e por que ele não é o GitHub

Git é um caderno dentro da pasta: o que mudou, em que ordem, com uma frase dizendo o porquê. Esse caderno pode morar na própria pasta do pendrive. Não precisa de conta nem de internet. O GitHub é só um lugar na internet para guardar esse caderno quando as pessoas não estão com o mesmo pendrive.

O caderno passa a valer quando o grupo cresce e cada um mexe em casa. Aí a cópia da internet evita a folha contra folha. Até esse dia, Cursor no computador mais a pasta do pendrive é um jeito completo de trabalhar. A conexão do GitHub pode esperar.

> **Para o grupo**
>
> Na próxima reunião, abram a pasta do pendrive com Open Folder e leiam o seletor antes do pedido. Se alguém quiser “subir no GitHub” e a tela pedir Sign in, parem. O trabalho do dia está na pasta. A conta fica para quando o grupo decidir que precisa do terceiro lugar.

BÁSICO

Instalar, entrar e abrir um projeto

Baixe o app, entre na conta e abra uma pasta. O primeiro pedido deve ser pequeno.

Baixe o instalador em cursor.com/download. Mac precisa do macOS 12 ou mais novo. Windows precisa do Windows 10 ou mais novo. No Linux, o pacote apt ou dnf é melhor que o AppImage: ele cria ícone, atualiza sozinho e instala as ferramentas de linha de comando.

Linux, do jeito recomendado

Debian e Ubuntu:

```
curl -fsSL https://downloads.cursor.com/keys/anysphere.asc | gpg --dearmor | sudo tee
/etc/apt/keyrings/cursor.gpg > /dev/null
echo "deb [arch=amd64,arm64 signed-by=/etc/apt/keyrings/cursor.gpg]
https://downloads.cursor.com/aptrepo stable main" | sudo tee
/etc/apt/sources.list.d/cursor.list > /dev/null
sudo apt update
sudo apt install cursor
```

Fedora e RHEL usam o repositório yum em `https://downloads.cursor.com/yumrepo` e, em seguida, `sudo dnf install cursor`. O AppImage existe para quem precisa de um arquivo portátil: dê permissão de execução e abra o arquivo.

Entrar e abrir a pasta

1. Abra o Cursor e entre na conta.
2. Abra a pasta do projeto, não um arquivo solto. O agente precisa ver o conjunto.
3. Espere a indexação começar. Ela permite a busca por significado, além da busca por texto.
4. Se o projeto já usa Git, o painel de controle de código-fonte mostra o que mudou. Use isso desde o primeiro dia.

> **Pratique agora**
>
> Abra um projeto pequeno, seu ou de exemplo. Não peça uma funcionalidade ainda. No próximo capítulo você vai reconhecer a tela; no seguinte, vai pedir uma explicação.

> **Conta e plano**
>
> Há planos individuais, de time e empresariais. O que muda é cota, administração e alguns recursos, como regras obrigatórias do time. O jeito de editar código é o mesmo.

BÁSICO

A tela, peça por peça

Editor no centro, agente na lateral, terminal embaixo. Quatro atalhos organizam o resto.

Se você já usou o VS Code, o miolo é familiar: à esquerda ficam explorador, busca, Git e extensões; no centro, os arquivos; embaixo, o terminal. O que o Cursor acrescenta é o painel do agente e um layout só para ele.

Atalho	Para que serve
`Cmd+I` ou `Cmd+L`	Abre ou fecha o painel lateral do agente.
`Cmd+E`	Alterna o layout do agente, quando você quer a conversa em primeiro plano.
`Cmd+.`	Abre o menu de modos.
`Cmd+Shift+J`	Abre as configurações do Cursor.
`Cmd+,`	Abre as configurações gerais do editor.
`Cmd+Shift+P`	Paleta de comandos. Se você não achar um botão, procure aqui.

O que olhar no painel do agente

- **Campo de texto.** É aqui que o pedido entra. `@` puxa contexto. `/` puxa um comando ou uma skill.
- **Seletor de modo.** Agent, Ask, Plan, Debug e, no navegador da janela de agentes, Design. `Shift+Tab` gira entre os modos.
- **Seletor de modelo.** Troca o modelo desta conversa daqui para a frente. `Cmd+/` percorre a lista.
- **Anel de contexto.** Mostra o quanto da janela do modelo já foi ocupada. Clique nele para ver a divisão: sistema, ferramentas, regras, skills, MCP, subagentes e a conversa.

Há também voz, no microfone do campo ou com `Cmd+Shift+Space`. Fale o pedido, leia a transcrição e só então envie.

> **Não decore a tela inteira**
>
> No primeiro dia, basta abrir o agente, saber onde está o modo e saber onde está o modelo. O resto aparece quando a tarefa pede.

BÁSICO

Tab, a sugestão enquanto você escreve

O Tab completa o que você está digitando. Aceite a sugestão inteira ou só a próxima palavra.

Enquanto você escreve, o Cursor oferece uma continuação em cinza. Essa ferramenta se chama Tab. Ela serve para o trecho que você já entendeu: o corpo de uma função, um teste parecido com o de cima, um campo que faltou.

Atalho	Efeito
`Tab`	Aceita a sugestão.
`Cmd+→`	Aceita só a próxima palavra.
`Esc`	Dispensa a sugestão e devolve o controle a você.

Quando confiar e quando não

Aceite quando a sugestão é o que você ia digitar. Recuse quando ela inventa um nome de função, muda um contrato ou repete um erro que já está no arquivo. O Tab não lê as regras do projeto. Uma regra em `.cursor/rules` não muda o que o Tab propõe. Quem obedece regra é o Agent do chat.

> **Pratique agora**
>
> Abra um arquivo que você conhece. Comece a escrever uma função pequena e pare no meio. Aceite com Tab se estiver certo. Se estiver quase certo, aceite palavra por palavra com `Cmd+→` e corrija o resto na mão.

BÁSICO

Edição inline

Selecione um trecho, descreva a mudança e receba um diff só naquele lugar.

A edição inline é o meio do caminho entre digitar sozinho e abrir uma conversa. Você marca o código, diz o que quer e o Cursor propõe a troca ali.

1. Selecione o trecho. Se não selecionar nada, o comando vale para a região em que está o cursor.
2. Aperte `Cmd+K`.
3. Escreva um pedido curto: “extraia esta validação para uma função” ou “traduza estas mensagens para português”.
4. `Enter` envia. `Option+Enter` faz uma pergunta rápida, sem editar.
5. `Cmd+Shift+Backspace` cancela.

`Cmd+Shift+K` alterna o foco do campo e também manda a seleção para a edição. Dentro do terminal, `Cmd+K` abre outra coisa: uma barra que gera um comando. `Cmd+Enter` executa esse comando e `Esc` apenas o aceita no terminal, sem rodar.

> **Regra que não entra aqui**
>
> As regras de usuário, aquelas preferências globais do chat, não valem para a edição inline. Se o tom, o idioma ou um padrão do projeto precisarem ser obedecidos, use o Agent.

> **Pedido que funciona neste modo**
>
> “Troque este loop por `map` e mantenha o mesmo resultado quando a lista vier vazia.” Um pedido de arquivo inteiro, ou de outro arquivo, já é trabalho de Agent.

BÁSICO

A primeira conversa com o Agent

Peça um mapa do projeto antes de pedir uma mudança. Enquanto ele trabalha, a fila de mensagens não corta o que está no meio.

Abra o painel com `Cmd+I`. A primeira mensagem útil, num projeto que você não domina, é esta:

```
Explique este projeto. Aponte os pontos de entrada, os módulos principais e o que eu
deveria ler antes de alterar qualquer coisa.
```

O agente busca, abre arquivos e devolve um mapa. Isso é mais rápido do que passear pasta por pasta. Quando o mapa fizer sentido, peça três melhorias pequenas e seguras, com o custo de cada uma, e espere você escolher. Só então peça para implementar a escolhida.

Enquanto a resposta está saindo

Atalho	Efeito
`Enter`	Enfileira a mensagem. Ela espera a tarefa atual terminar.
`Cmd+Enter`	Envia na hora e junta o texto à última mensagem sua.
`Enter` duas vezes	Encaminha um ajuste no próximo uso de ferramenta, sem jogar fora o trabalho que já está em andamento. Esse comportamento está na web e está chegando à janela de agentes.
`Cmd+Shift+Backspace`	Cancela a geração.
`Cmd+N` ou `Cmd+R`	Conversa nova.
`Cmd+T`	Nova aba de conversa.
`Cmd+[` e `Cmd+]`	Conversa anterior e próxima.
`Cmd+W`	Fecha a conversa.

`Tab`, com o campo do chat em foco e sem uma sugestão de código na frente, circula entre mensagens. `Esc` tira o foco do campo.

> **Conversa nova ou a mesma?**
>
> Continue na mesma conversa enquanto estiver na mesma função. Abra outra quando mudar de assunto, quando o agente repetir o mesmo erro, ou quando a resposta começar a misturar tarefas velhas. Conversa longa acumula ruído.

BÁSICO

Dar contexto com @

Marque o arquivo quando você sabe qual é. Não marque a pasta inteira por precaução.

Digite `@` no campo do agente. O Cursor lista o que dá para anexar. Você não precisa marcar tudo: se o pedido disser “o fluxo de login”, o agente procura. Marque arquivo quando quiser cravar o alvo. Arquivo irrelevante compete com o arquivo certo.

Menção	O que entra na conversa
`@arquivo` ou `@pasta/`	O arquivo, ou a pasta. Depois de escolher a pasta, `/` desce mais um nível.
`@Terminals`	A saída do terminal.
`@Chats`	Um pedaço de uma conversa anterior, sem copiar o chat inteiro.
`@Commit`	O diff do que ainda não foi commitado.
`@Branch`	O diff da branch em relação à principal. Serve para “o que eu estou fazendo?” e para “revise esta branch”.
`@Browser`	Contexto do navegador embutido.

Seleção, área de transferência e imagem

- Com código selecionado, `Cmd+Shift+L` manda a seleção para o chat. `Cmd+L` abre uma conversa nova já com essa seleção.
- Copie um trecho ou um log e cole com `Cmd+V` para anexá-lo como contexto. `Cmd+Shift+V` cola o texto cru no campo.
- Arraste uma imagem, ou cole um print com `Cmd+V`, para mostrar uma tela, um erro ou um mockup.

> **Pratique agora**
>
> Peça: “Explique só esta função” com a função selecionada e `Cmd+Shift+L`. Depois peça a mesma coisa sem seleção, citando o nome. Compare o quanto cada resposta acerta.

BÁSICO

Ler o diff e decidir

O agente termina quando você aceita, rejeita ou pede ajuste. O diff é a revisão, não um detalhe.

Quando o Agent edita, as mudanças aparecem como diff: o que saiu e o que entrou. Leia o diff com a mesma atenção que daria a um colega. O texto da resposta explica a intenção. O diff mostra o que realmente mudou. Os dois divergem com frequência.

Situação	Atalho
Aceitar todas as mudanças sugeridas	`Cmd+Enter`
Rejeitar todas	`Cmd+Shift+Backspace`

Dá para aceitar arquivo por arquivo, ou hunk por hunk, nos botões do próprio diff. Faça isso quando uma parte estiver boa e outra não. Rejeitar tudo e reescrever o pedido costuma ser mais limpo do que tentar consertar cinco erros com uma frase genérica do tipo “arrume”.

O que conferir antes de aceitar

1. A mudança faz o que você pediu, e só isso?
2. Algum arquivo que você não queria tocar entrou no diff?
3. Teste, tipo ou lint do projeto foi executado? Se não, peça: “rode a checagem que este projeto já usa”.
4. Há segredo, comentário de debug ou arquivo gerado no meio da mudança?

> **Você assina o código**
>
> Aceitar o diff é uma decisão sua. O agente não vira autor no lugar de quem faz o commit.

BÁSICO

Os atalhos que valem decorar

Doze atalhos cobrem o básico. O mapa completo fica no fim do livro.

Decore estes. Os outros você busca na paleta até virarem hábito.

Atalho	Ação
`Cmd+I`	Painel do agente
`Cmd+K`	Edição inline no editor, ou gerar comando no terminal
`Tab`	Aceitar sugestão do Tab
`Shift+Tab`	Girar o modo do agente
`Cmd+/`	Trocar de modelo
`Cmd+Shift+L`	Mandar a seleção para o chat
`Cmd+Enter`	Enviar na hora, ou aceitar as mudanças quando elas estão na tela
`Cmd+Shift+Backspace`	Cancelar a geração, ou rejeitar as mudanças
`Cmd+Shift+P`	Paleta de comandos
`Cmd+Shift+J`	Configurações do Cursor
`@` e `/`	Contexto e comandos, dentro do campo do agente
`Cmd+N`	Conversa nova

A lista oficial, inclusive o que dá para remapear, está em Keyboard Shortcuts.

BÁSICO

Do pedido ao resultado

Um caminho curto para a primeira mudança que você confiaria em deixar no projeto.

1. Abra a pasta do projeto e entre na conta.
2. Peça o mapa do código, como no capítulo da primeira conversa. Leia. Se um módulo importante não aparecer, pergunte por ele.
3. Peça três melhorias pequenas e escolha uma. Bom primeiro alvo: texto de interface, um bug visível, um teste que falta numa função pura.
4. Descreva o resultado e como verificar. Exemplo: “No formulário de busca, o botão fica desabilitado enquanto a lista está vazia. Não mude o estilo dos outros botões. Rode o teste desta tela.”
5. Deixe o agente trabalhar. Se lembrar de um detalhe no meio, `Enter` enfileira. Se o detalhe muda o pedido atual, `Cmd+Enter` manda na hora.
6. Leia o diff. Aceite o que estiver certo. Rejeite o resto e diga o que faltou, com o arquivo na frase.
7. Peça a checagem que o projeto já tem: teste, tipos ou lint. Olhe a saída, não só o “passou” da resposta.

Mudança que atravessa vários arquivos, ou que você ainda não sabe como desenhar, não entra neste fluxo. Ela espera o modo Plan, no nível intermediário.

> **O pedido mínimo que já é bom**
>
> Resultado observável, limite do que não pode mudar, e um comando de verificação. Três linhas bastam.

BÁSICO

Pratique o básico

Cinco exercícios curtos. Faça no seu projeto, não só de cabeça.

1. Abra o agente e peça o mapa do projeto em uma conversa nova. Anote dois arquivos que você não conhecia.
2. Selecione uma função e use `Cmd+Shift+L`. Peça uma explicação de dez linhas, sem edição.
3. Na mesma função, use `Cmd+K` para um ajuste local: nome, comentário ou um caso de borda. Aceite ou rejeite olhando o diff, não a mensagem.
4. Escreva um teste ou um trecho repetido e use o Tab. Aceite uma sugestão inteira e, na seguinte, só a próxima palavra.
5. Peça uma mudança pequena com resultado, limite e verificação. Antes de aceitar, responda: algum arquivo surpresa entrou no diff?

Se os cinco saíram sem você procurar botão no escuro, o básico está feito. O intermediário começa quando a tarefa deixa de caber num único pedido.

> **Erro comum neste ponto**
>
> Abrir uma conversa infinita para assuntos diferentes. Uma conversa por tarefa. O histórico continua lá; você não precisa carregar tudo para a frente.

INTERMEDIÁRIO

Os modos do Agent

Agent faz. Ask só lê. Plan combina antes de escrever. Debug mede o bug. Design aponta na tela.

O seletor de modo fica no campo do agente. `Shift+Tab` gira os modos. `Cmd+.` abre o menu. Use o modo certo antes de escrever um pedido longo no modo errado.

Agent

É o padrão. Busca, edita, roda comando e devolve diff. Serve para tarefa que você já sabe descrever e que não precisa de um desenho prévio.

Ask

Ask lê e responde. Não edita. Use para entender um fluxo, comparar duas abordagens ou revisar uma ideia antes de autorizar código. No CLI, o comando é `/ask` e a flag é `--mode=ask`.

Plan

Aperte `Shift+Tab` até o Plan, ou deixe o Cursor sugerir o modo quando o pedido parece grande. O agente pesquisa o repositório, faz perguntas, escreve um plano e espera você aprovar. O plano abre como Markdown. Você pode cortar passo, trocar arquivo e acrescentar o que ele não viu.

Por padrão o plano fica na sua pasta de usuário. *Save to workspace* grava em `.cursor/plans/`, para o time e para um agente futuro. Use Plan quando houver mais de um desenho possível, muitos arquivos, requisito frouxo ou decisão de arquitetura. Tarefa que você já fez dez vezes pode ir direto ao Agent.

Se a implementação sair torta, volte ao plano em vez de empilhar correções. Desfaça a mudança, aperte o plano e rode de novo. Em geral fica mais limpo.

Debug

Debug serve para bug que você consegue reproduzir e não consegue explicar: condição de corrida, vazamento, regressão. A ordem é fixa. O agente levanta hipóteses, adiciona logs que vão para um servidor local da extensão, pede para você reproduzir, lê os logs, corrige pouco código e, no fim, tira a instrumentação. Siga os passos de reprodução à letra. Diga o que deveria acontecer e o que acontece.

Design

Design Mode fica no navegador da janela de agentes, não no editor comum. Abra o navegador e alterne com `Cmd+Shift+D`. Você clica num elemento, seleciona vários, desenha uma região com `Shift` e arraste, ou fala. `Cmd+L` manda o elemento ao chat. `Option+clique` manda o elemento ao campo. O agente recebe o componente e um print da tela. Dá para enviar o próximo ajuste antes de o anterior terminar. A própria documentação recomenda o Composer 2.5 para esse fluxo de interface.

> **Escolha em uma linha**
>
> Dúvida sem edição: Ask. Tarefa clara: Agent. Tarefa grande ou ambígua: Plan. Bug que precisa de execução: Debug. Ajuste visual na aplicação rodando: Design.

INTERMEDIÁRIO

Como pedir

O agente acha arquivo sozinho. O que ele não inventa bem é o critério de pronto.

Um pedido bom tem quatro partes, nem sempre em quatro parágrafos: o resultado, o limite, o critério de pronto e o contexto que só você tem.

```
No checkout, o frete grátis deve aparecer quando o subtotal passar de R$ 199.
Não altere o cálculo de imposto.
Cubra os casos 198,99 e 199,00 no teste que já existe para o carrinho.
O valor do limite está em config/frete.ts.
```

O que deixar de fora

- A pasta inteira marcada com `@`, “para garantir”.
- O histórico de outra tarefa colado no campo. Use `@Chats` se precisar de um pedaço.
- Estilo literário: “seja profissional e robusto”. Diga a restrição real.
- Pedido duplo sem ordem. “Refatore e também explique” vira diff misturado com aula. Peça um, depois o outro.

Contexto que enche

Cada conversa divide uma janela limitada com o modelo. Arquivo, ferramenta, regra, skill e MCP ocupam lugar. Quando a janela aperta, o Cursor resume o começo da conversa. O anel ao lado do campo mostra a divisão. Conversa que já foi resumida várias vezes perde foco: abra outra e aponte o que importa.

Troque de modelo no meio da conversa se a exploração pediu um modelo rápido e a implementação pede um mais capaz. A troca vale daquele ponto em diante. O padrão fica em Cursor Settings, em Models.

> **Reescreva este pedido**
>
> “Melhore o login.” Vira: “Depois de três senhas erradas, a tela mostra a mensagem que já existe em `erros.login.bloqueado` e não chama a API de novo por 30 segundos. Não mude o layout. O teste está em `login.test.ts`.”

INTERMEDIÁRIO

Regras e AGENTS.md

Regra é instrução persistente para o Agent. Ela não muda o Tab nem a edição inline.

O modelo não lembra o projeto entre conversas. Regra é o que entra no começo do contexto toda vez que se aplica. Existem quatro origens: regras do projeto em `.cursor/rules`, regras suas em Customize → Rules, regras do time no painel, e `AGENTS.md`.

O arquivo .mdc

Regra de projeto é um arquivo `.mdc`. Um `.md` solto dentro de `.cursor/rules` é ignorado, porque não tem o frontmatter. Se você quer Markdown simples, use `AGENTS.md` na raiz ou numa subpasta.

Crie pela conversa com `/create-rule`, ou em Customize → Rules → Add Rule. O tipo da regra sai da combinação de três campos:

alwaysApply	description	globs	Quando entra
true	tanto faz	tanto faz	Em toda conversa. Os outros campos são ignorados.
false	vazio	preenchido	Quando um arquivo que casa com o padrão está no contexto.
false	preenchida	vazio	Quando o agente julga a descrição relevante.
false	vazio	vazio	Só se você mencionar a regra com `@`.

```
---
globs: src/components/**/*.tsx
alwaysApply: false
---

- Exporte o componente pelo nome, não como default.
- Estilo fica no CSS module ao lado do componente.
- Leia @src/components/Button.tsx antes de criar um componente novo.
```

Vários padrões no mesmo campo se separam por vírgula: `docs/**/*.md, docs/**/*.mdx`. Uma menção `@arquivo` dentro da regra aponta o caminho. O conteúdo desse arquivo não é colado no prompt; o agente lê se precisar. O que tem de estar sempre no contexto precisa estar escrito no corpo da regra.

AGENTS.md

Serve para instrução curta, sem metadado. Dá para ter um `AGENTS.md` na raiz e outros em `frontend/` ou `backend/`. O da pasta mais específica prevalece e se soma aos pais.

O que não colocar

- Manual de estilo inteiro. Isso é trabalho de linter.
- Todo comando que o agente já conhece, como `npm test` genérico.
- Caso raro. Regra boa nasce do erro que se repetiu.
- Cópia de um arquivo do repositório. Aponte o exemplo canônico.

Mantenha cada regra abaixo de umas 500 linhas e divida por assunto. Versione no Git. A precedência, quando há conflito, é regra do time, depois regra do projeto, depois regra de usuário. Regra não altera o Tab. Regra de usuário não entra na edição inline (`Cmd+K`).

> **Primeira regra que vale a pena**
>
> Escreva só os comandos reais do repositório e um ponteiro para o arquivo modelo. Acrescente uma linha no dia em que o agente repetir o mesmo erro.

INTERMEDIÁRIO

Modelos, contexto e o que não enviar

Modelo rápido para o rotineiro, modelo forte para o emaranhado. Segredo não entra no chat.

O seletor de modelo fica no campo do agente. `Cmd+/` percorre os modelos. `Cmd+Option+/` é o outro atalho de troca documentado. A mudança vale para o restante da conversa. Há modelos da Anthropic, OpenAI, Google, Cursor, xAI, Moonshot, Meta e outros. A lista e o preço moram em Models & Pricing e mudam seguido. Não decore tabela de preço.

Como escolher sem planilha

- Edição local, renomear, teste espelhado, texto de interface: modelo mais rápido.
- Vários arquivos, desenho de API, bug que atravessa camadas: modelo mais capaz.
- Interface na janela de agentes, no fluxo do Design Mode: a documentação recomenda o Composer 2.5.
- Alguns modelos só aparecem em Max Mode nos planos antigos cobrados por request, e janela longa custa mais. Se o seletor avisar, leia o aviso antes de deixar a conversa crescer.

Indexação e o que o agente não deve ver

`.cursorignore` esconde arquivos do agente, no mesmo espírito de um ignore. Use para segredo local, dump, pasta gerada enorme e dado que não deveria ir para o modelo. O sandbox também trata `.cursorignore` como caminho protegido. Isso não substitui não colar token no chat.

Em Settings → Agents → Approvals & Execution dá para restringir leitura fora do workspace (modo Workspace, a partir do Cursor 3.23). O padrão System deixa o agente ler fora da pasta. Em projeto com arquivo sensível ao lado, Workspace é o ajuste mais sóbrio.

> **Não cole isto**
>
> Chave de API, senha, cookie de sessão, dump de produção, documento de cliente, conteúdo de `.env`. Descreva o nome da variável. Deixe o valor no ambiente.

Privacidade e retenção dependem do plano e do modelo. Alguns modelos de fronteira exigem aceite explícito de retenção para prevenção de abuso, mesmo sem uso para treino. Leia o aviso da conta antes de ligar um modelo novo num repositório privado.

INTERMEDIÁRIO

Terminal, navegador, imagem e voz

O agente roda comando e abre a aplicação. Você decide o quanto isso acontece sem perguntar.

O Agent usa o terminal de verdade: instala dependência, roda teste, sobe o servidor. Também usa um navegador para olhar a página que ele mesmo alterou. Imagem e voz entram pelo campo: arraste o print, cole com `Cmd+V`, ou dite no microfone e revise o texto antes de enviar.

Quem autoriza o comando

Em Settings → Agents → Approvals & Execution há três modos de execução:

Modo	Comportamento
Auto-review	O recomendado. O que está na lista permitida roda. Outros comandos de shell entram no sandbox quando dá. O resto passa por um classificador. O classificador erra: pode liberar o que você bloquearia.
Allowlist	Só o que você listou roda sem perguntar. Comportamento estável, lista curta.
Run Everything	Toda ferramenta roda sem pergunta e sem sandbox. Só se você aceitar esse risco.

O sandbox deixa o comando trabalhar na pasta do projeto e corta rede e caminho sensível, inclusive `.git/config` e `.git/hooks`. No Linux ele usa Landlock e seccomp; kernel antigo faz o Cursor voltar a pedir aprovação. Há ainda proteção extra para navegador, para apagar arquivo e para escrever fora do workspace.

Instrução em português cabe em `.cursor/permissions.json` ou em `~/.cursor/permissions.json`, no campo `block_instructions`. Exemplo de frase: “Todo comando da AWS CLI precisa da minha aprovação.” Dá para pedir ao próprio agente que grave isso. O arquivo do time, quando existe, ganha da sua máquina.

> **No terminal, sem abrir o chat**
>
> `Cmd+K` gera um comando a partir de uma frase. `Cmd+Enter` executa. `Esc` só coloca o comando na linha.

Voltar atrás

Se a conversa mostrar restauração de um ponto anterior, use para desfazer um trecho ruim. O Git continua sendo a rede que não depende da interface. Commit pequeno, antes de um pedido arriscado, custa pouco.

INTERMEDIÁRIO

Git sem perder o controle

O painel de Git é o do editor. O agente entra quando você entrega o diff de propósito.

Git é o caderno das mudanças. GitHub é um lugar na internet para esse caderno. Dá para usar o Cursor na pasta do pendrive sem essa conta. Esse caminho está em Pendrive e GitHub. O que segue aqui é para quando o caderno já existe.

Fonte, branch, stage e commit ficam no painel de controle de código-fonte. O agente não precisa commitar para você. Muitas vezes é melhor ele não commitar: o commit é o momento em que você afirma que leu o diff.

Contexto de Git que vale mencionar

- `@Commit`, diff do que ainda não entrou no commit: “revise só isto”.
- `@Branch`, diff contra a branch principal: “resuma o que esta branch faz” ou “procure regressão neste conjunto”.

Um ciclo que não vira bagunça

1. Branch nova para a tarefa. Nome que um colega entende.
2. Pedido no Agent ou no Plan.
3. Você lê o diff no painel, não só o resumo do chat.
4. Teste. Se falhar, cole a saída com `@Terminals` ou `Cmd+V` e peça a correção da falha, não uma reescrita.
5. Commit seu, com mensagem que diz o porquê.

> **O que não deixar o agente fazer sozinho**
>
> Push forçado, commit de `.env`, alteração de hook do Git, rebase de branch compartilhada. O sandbox já protege parte disso. A sua regra e a sua leitura protegem o resto.

INTERMEDIÁRIO

Skills, comandos / e MCP

Barra chama um procedimento. MCP liga o agente a um sistema de fora do repositório.

Digite `/` no campo. Aparecem comandos e skills. `Enter` gruda a skill naquela mensagem. `Option+Enter`, ou *Use as Mode*, liga a skill como modo personalizado: ela permanece em toda a conversa até você sair. Isso serve para um checklist de revisão ou um playbook de teste, não para um comando de uma tacada só.

Skills que já vêm prontas

Comando	Uso
`/create-rule`	Cria uma regra `.mdc` com o frontmatter certo.
`/create-skill`	Cria uma skill, com `SKILL.md`.
`/create-hook`	Cria um hook e atualiza o `hooks.json`.
`/create-subagent`	Cria um subagente com papel próprio.
`/review`, `/review-bugbot`, `/review-security`	Revisão geral, caça a bug, ou revisão de segurança.
`/update-cursor-settings`	Acha e altera a configuração certa do Cursor ou do editor.

Regra entra sempre que o tipo dela manda. Skill entra quando o agente julga que cabe, ou quando você chama com `/`. Por isso skill não entope a janela do mesmo jeito que uma regra gigante. Dá para impedir a invocação automática com `disable-model-invocation: true` no frontmatter: aí ela só entra se você chamar.

MCP

MCP (Model Context Protocol) liga o agente a ferramenta e dado de fora: issue, banco, log, documentação interna. Instale em Customize ou descreva o servidor em `mcp.json`. O protocolo cobre ferramentas, prompts, recursos e, no Cursor, interfaces interativas devolvidas pela ferramenta.

O agente chama a ferramenta quando o pedido pede. Você também pode citar o nome. Ligue e desligue servidores em Customize. No modo Auto-review, ferramenta MCP que não está na lista passa pelo classificador. Não coloque segredo no corpo do `mcp.json` que vai para o Git. Time e empresa podem permitir ou bloquear servidores no painel.

> **Quando criar cada coisa**

> Erro repetido de estilo ou de comando: regra curta. Procedimento com passos e talvez um script: skill. Sistema fora do repositório: MCP. Ainda não crie hook nem subagente; isso é o nível avançado.

INTERMEDIÁRIO

Pratique o intermediário

Seis exercícios que mudam o jeito de pedir, não só o cardápio de botões.

1. Pegue uma tarefa de verdade com mais de um arquivo. Mude para Plan com `Shift+Tab`. Responda as perguntas. Corte um passo inútil do plano antes de construir. Salve o plano no workspace.
2. Num bug que você consegue repetir, use Debug. Siga a reprodução pedida. Confira se a instrumentação saiu no fim.
3. Escreva uma regra `.mdc` com `alwaysApply: false` e um `globs` estreito. Peça uma edição num arquivo que casa e noutra que não casa. Veja a diferença.
4. Abra `AGENTS.md` na raiz com três linhas: comando de teste, comando de tipos, e o arquivo que serve de modelo.
5. Em Approvals & Execution, identifique o modo atual. Se estiver em Run Everything num projeto que não é uma caixa de areia, volte para Auto-review.
6. Chame `/review` num diff que você mesmo acabou de fazer. Trate o comentário como revisão de colega: aceite o que for bug, dispense o que for gosto.

O avançado começa quando o trabalho não cabe mais na sua máquina aberta: agente na nuvem, revisão no pull request, vários agentes, e um freio automático quando o loop erra.

AVANÇADO

Cloud Agents

Um agente numa máquina remota clona o repositório, abre branch e devolve pull request. Você pode fechar o notebook.

Cloud Agent roda fora do seu editor, numa máquina isolada. Ele clona o repositório, trabalha numa branch e abre pull request. Use isto quando você escolher a nuvem. Para não cair nela sem querer, o procedimento está no básico, em No seu computador, não na nuvem.

Dá para começar de vários lugares:

- No Cursor, escolha Cloud no menu abaixo do campo do agente.
- Na web, em cursor.com/agents.
- No Slack ou no Linear, mencionando `@Cursor`.
- No GitHub ou no Bitbucket Cloud, comentando `@cursor` num pull request ou numa issue.
- Pela API de Cloud Agents.

A máquina tem ambiente de desktop. O agente usa mouse e teclado para abrir o que construiu, inclusive o navegador. Por isso um cloud agent consegue verificar tela, não só teste de unidade. Os modos de aprovação da sua máquina local não se aplicam: na nuvem ele não para para pedir ok a cada comando, porque a máquina é dele.

Ambiente, segredo e MCP

Conecte a conta do Git (GitHub, GitLab, Azure DevOps ou Bitbucket), escolha os repositórios e grave segredo no ambiente, não no prompt. Ambiente mal preparado é a causa mais comum de agente que "não acha o projeto". Build de ambiente e snapshot ficam na configuração do cloud agent; vale a pena deixar instalação e teste reproduzíveis antes de delegar tarefa grande.

MCP na nuvem aceita HTTP e stdio. HTTP é o caminho recomendado: a credencial não fica dentro da máquina do agente, e a chamada passa pelo backend. Stdio roda dentro da VM, então o agente enxerga a configuração. SSE não é suportado nesse modo. O dropdown de MCP em cursor.com/agents liga servidores pessoais. O time configura os compartilhados no painel, em Plugins & MCPs.

Regras de usuário, do time e do repositório (`.cursor/rules` e `AGENTS.md`) valem para o cloud agent. Hook de usuário em `~/.cursor/hooks.json` não sobe junto: a VM não tem a sua pasta pessoal. Hook do repositório, em `.cursor/hooks.json`, sobe.

> **O que delegar**
>
> Tarefa com critério de pronto verificável: teste que falha, issue bem escrita, migração com plano já aceito. Tarefa em que você ainda não sabe o que quer continua no Plan, na sua máquina, até o

> plano ficar curto o bastante para outra pessoa executar. O agente remoto é essa outra pessoa.

AVANÇADO

A linha de comando

O mesmo agente, no terminal. Serve para script, máquina sem interface e quem já vive no shell.

No macOS, Linux e Windows com WSL:

```
curl https://cursor.com/install -fsS | bash
agent --version
```

No Windows nativo, o instalador é PowerShell: `irm 'https://cursor.com/install?win32=true' | iex`. Inclua `~/.local/bin` no `PATH` se o comando `agent` não aparecer. Atualização manual: `agent update`. O CLI tenta se atualizar sozinho.

Modos e revisão

- `Shift+Tab` gira Agent, Plan e Ask. `/plan` e `--mode=plan` abrem o Plan. `/ask` e `--mode=ask` só leem. `/debug` entra no Debug.
- `agent -p` ou `--print` responde sem interface. É o caminho para script e CI. Trate isso como automação com permissão de verdade: o modo de execução e o sandbox continuam valendo.
- `Ctrl+R` revisa as mudanças. `i` acrescenta uma instrução. As setas trocam de arquivo.
- `Shift+Enter` quebra linha em vários terminais modernos. No tmux, use `Ctrl+J`. `Ctrl+D` sai, e pede a segunda vez.

Comandos úteis no dia a dia: `/model`, `/mcp list`, `/sandbox`, `/run-everything` (o alias é `/auto-run`), `/max-mode` nos planos antigos, `/goal` para um objetivo que permanece até terminar. `/loop` repete um pedido ou uma skill; se você não fixar o intervalo, o agente escolhe quando acordar. Pare o loop quando o objetivo acabar.

Skill vira modo persistente também no CLI: no menu `/`, `Option+Enter` mantém a skill ativa.

> **Headless não é menos perigoso**
>
> Um `agent -p` dentro de CI enxerga o que o job enxerga. Segredo de CI, rede e escrita no repositório precisam da mesma política que você usaria para um script qualquer. Comece em Ask ou com sandbox, não com Run Everything.

AVANÇADO

Subagentes

Um subagente tem contexto próprio. O pai recebe o resultado, não o barulho do caminho.

Subagente é um agente delegado. Ele não vê o histórico do pai. O pai escreve um pedido completo, o subagente trabalha numa janela limpa e devolve uma mensagem final. Isso serve para pesquisa longa, trabalho em paralelo e checagem independente. Serve pouco para “formata este arquivo”: isso é skill.

Dois já existem e o Agent os chama sozinho. **Explore** vasculha o repositório com um modelo mais rápido, em buscas paralelas, e não despeja essa leitura na sua conversa. **Browser** fica com o DOM e os prints e devolve só o que importa.

Modo	Efeito
Foreground	O pai espera. Use quando o próximo passo depende da resposta.
Background	O pai segue. Use para trabalho longo ou paralelo.

Criar o seu

Arquivo de projeto em `.cursor/agents/nome.md`. Arquivo seu, para todos os projetos, em `~/.cursor/agents/`. Também há pastas compatíveis `.claude/agents` e `.codex/agents`; em conflito de nome, `.cursor/` ganha, e o do projeto ganha do seu usuário. `/create-subagent` escreve o arquivo.

```
---
name: verificador
description: Confere se a tarefa terminou. Use depois de uma implementação.
model: inherit
readonly: true
---

Rode os testes do escopo pedido.
Diga o que passou, o que falhou e o que não foi verificado.
Não altere arquivo.
```

`model: inherit` usa o modelo do pai. Um id específico força outro modelo. `readonly: true` tira edição e comando que muda estado. `is_background: true` não bloqueia o pai. A descrição é o que o agente lê para decidir se delega: escreva quando usar, não um slogan.

> **Skill ou subagente**
>
> Um passo repetível, numa janela só: skill. Pesquisa que suja o contexto, trabalho paralelo ou uma segunda opinião com contexto limpo: subagente.

AVANÇADO

Hooks

Um script seu roda antes ou depois de um passo do agente. Ele pode observar, barrar ou pedir continuação.

Hook é processo definido em `.cursor/hooks.json` no projeto, em `~/.cursor/hooks.json` na sua máquina, ou instalado por um plugin. O script fala JSON pelo stdio. `/create-hook` cria o esqueleto. Há uma aba Hooks em Customize e um canal de saída para ver erro.

Onde eles disparam

- Sessão: `sessionStart`, `sessionEnd`.
- Ferramenta: `preToolUse`, `postToolUse`, `postToolUseFailure`.
- Shell e MCP: `beforeShellExecution`, `afterShellExecution`, `beforeMCPExecution`, `afterMCPExecution`.
- Arquivo: `beforeReadFile`, `afterFileEdit`.
- Conversa: `beforeSubmitPrompt`, `preCompact`, `stop`, `afterAgentResponse`, `afterAgentThought`.
- Subagente: `subagentStart`, `subagentStop`.
- Tab, à parte: `beforeTabFileRead`, `afterTabFileEdit`.
- Aplicação: `workspaceOpen`, ao abrir a pasta. Não depende de uma conversa.

Os hooks de Agent valem para o chat e para a edição com `Cmd+K`. Os de Tab valem só para a sugestão inline. Cloud Agent roda os hooks de comando do repositório, inclusive shell, arquivo, ferramenta e o ciclo `stop`. Não roda hook de Tab, `workspaceOpen`, hook baseado em prompt, nem o `hooks.json` da sua pasta de usuário.

Um uso que presta

O hook `stop` pode devolver `followup_message` e fazer o agente continuar, com um teto de voltas. O caso sério é “rode o teste e siga até passar”, com um arquivo de rascunho que marca `DONE` e um máximo de iterações. Sem teto, o loop gasta cota e pode alargar o diff. O mesmo efeito de repetição, com menos encanamento, também existe na skill `/loop`.

> **Hook é código de produção**
>
> Ele vê caminho de arquivo e pode bloquear ferramenta. Revise como revisaria um script de CI. Não coloque segredo no script versionado. Não use hook para esconder do revisor uma ação que deveria aparecer no diff.

AVANÇADO

Bugbot, revisão e automações

A revisão automática olha o pull request. A automação dispara trabalho sem você abrir o editor.

Bugbot lê o diff do pull request e comenta bug, segurança e qualidade, com sugestão de correção. Liga-se em Automations, depois de conectar o repositório no painel: GitHub, GitLab, Bitbucket, Azure DevOps ou Origin. Ele roda a cada atualização do PR, ou só quando alguém comenta `cursor review` ou `bugbot run`. Lê os comentários que já existem para não repetir. *Fix in Cursor* abre o ponto no editor. *Fix in Web* abre em cursor.com/agents.

No GitHub o check se chama `Cursor Bugbot`. Achado, por padrão, publica conclusão neutra, não falha. Exigir o check na proteção da branch garante que ele rodou; para bloquear merge por causa de achado, é preciso a opção de falhar quando ainda há comentário sem resolver, se o seu plano tiver essa opção. Há também `/review-bugbot` e `/review-security` dentro do editor, para a mudança que ainda não virou PR.

Arquivo opcional no repositório

`.cursor/config/bugbot.yaml` ajusta rascunho, frequência (`everyPush` ou `oncePerPr`), esforço da revisão e autofix (`disabled`, `newBranch` ou `existingBranch`). Campo ausente cai no padrão. Arquivo acima de 64 KB é ignorado. Não é obrigatório para começar.

Automações

A skill `/automate` cria automação disparada por horário, mensagem de Slack, evento de GitHub e outras fontes. `/autopilot` acompanha um pull request e trata comentário, conflito e check vermelho. Isso é o avançado de verdade: o gatilho deixa de ser você sentado no editor. Escreva o critério de pronto no texto da automação com o mesmo cuidado de um pedido Plan. Automação vaga produz diff vago em horário comercial dos outros.

> **Ordem de adoção**
>
> Primeiro Bugbot comentando, sem autofix. Você lê uma semana de comentários. Só então ligue correção automática, e ainda em branch nova, para o diff continuar visível.

AVANÇADO

Time, integrações e governança

Regra de empresa, MCP permitido e integração com o tracker. O agente passa a fazer parte do processo, não de uma preferência pessoal.

Plano Team e Enterprise administram regras no painel. Regra de time é texto, com glob opcional. Sem glob, vale em toda conversa, em todo repositório. Com glob, só quando um arquivo daquele padrão está no contexto. Precedência quando o texto briga: time, projeto, usuário. As aplicáveis se somam.

O admin pode exigir a regra. Aí o membro não desliga em Customize. Rascunho não se aplica até alguém ativar. Regra de IA não é controle de segurança. Segredo, permissão de produção e revisão humana continuam fora do prompt.

Onde o Cursor encontra o resto da empresa

- GitHub, GitLab, Azure DevOps e Bitbucket: repositório, Bugbot e cloud agent.
- Slack, Microsoft Teams, Linear, Jira e Notion: disparar ou acompanhar trabalho sem abrir o editor.
- JetBrains e Xcode: há integração documentada para quem não vive o dia inteiro no Cursor.
- Plugins e marketplace: regras e skills empacotadas. Importar um repositório de regras “no olho” não funciona; elas entram por plugin, com `.cursor-plugin/marketplace.json`.

Empresa ainda tem SSO, SCIM, controle de modelo, controle de MCP, política de rede, exportação OpenTelemetry e revisão de segurança. O capítulo útil para quem não é admin: se um modelo sumiu, se um MCP não conecta, ou se o Auto-review está cinza, a causa pode ser política do time, não a sua instalação. Auto-review, por exemplo, precisa que Claude 4.5 Haiku permaneça permitido, porque o classificador usa esse modelo como reserva.

> **O mínimo de um time pequeno**
>
> Uma regra curta obrigatória (comando de teste e o que nunca commitar), Bugbot no repositório principal, e MCP só dos sistemas que o time realmente usa. O resto é enfeite até doer a falta.

AVANÇADO

Quando o agente erra

A correção mais rápida quase nunca é a quinta mensagem na mesma conversa.

Sintoma e o que fazer, nesta ordem:

1. **Diff grande demais ou no lugar errado.** Rejeite. Reescreva o pedido com arquivo, limite e teste. Não diga só “não era isso”.
2. **A implementação não bate com o que você queria, e a tarefa era grande.** Volte ao plano. Desfaça. Ajuste o plano. Rode de novo.
3. **O mesmo erro pela terceira vez.** Conversa nova. A anterior já ensinou o modelo a insistir. Se o erro for de convenção do repositório, vire uma linha de regra.
4. **Bug que depende de rodar.** Saia do achismo e vá para Debug, com reprodução. Cole o erro real, via `@Terminals` ou o print.
5. **A resposta ignora um arquivo óbvio.** Marque com `@`. Se o arquivo nem deveria ser lido, ele provavelmente está fora do ignore e competindo, ou a conversa está cheia.
6. **O anel de contexto está no limite.** Abra outra conversa. Aponte o plano salvo, o diff da branch com `@Branch` e a regra. Não cole o chat velho.
7. **Comando perigoso ou surpresa.** Recuse o cartão de aprovação. Se isso se repetir, escreva uma `block_instructions` em `permissions.json`.
8. **Cloud agent entregou PR ruim.** Comente no PR com o critério que faltou, ou feche e rode de novo com um pedido mais curto. Não mergeie para “ir ajustando”.

> **A frase que recupera a conversa**
>
> “Pare. Não edite mais. Diga qual arquivo você acredita que contém a causa e qual teste prova isso.” Só então autorize a próxima edição.

AVANÇADO

Um jeito de trabalhar

Plano para o que é grande, agente para o que é claro, você para o diff e para o teste.

Quem usa o Cursor o dia inteiro não escreve prompt mais bonito. Separa o trabalho em tipos e não mistura os tipos na mesma conversa.

Tipo de trabalho	Onde fazer
Entender código alheio	Ask, ou um Agent só de leitura no primeiro pedido.
Mudança local que você já vê	Tab ou `Cmd+K`.
Função nova, vários arquivos	Plan, plano salvo no repositório, depois Agent.
Bug reproduzível	Debug, com o erro colado.
Ajuste de tela com o app aberto	Design Mode no navegador da janela de agentes.
Tarefa que pode terminar sem você	Cloud Agent, com teste que decide sozinho se passou.
PR dos outros, ou o seu antes do merge	Bugbot, mais a sua leitura do que o bot marcou como neutro.

O que deixar escrito no repositório

- `AGENTS.md` curto: como testar, como subir, o que não gerar.
- Uma regra por pasta que tem jeito próprio, com `globs`, apontando o arquivo modelo.
- Skill para o procedimento que você repetiria em voz alta para um colega novo.
- Subagente somente leitura para “confira se terminou”, quando a tarefa for longa o bastante para o agente se elogiar cedo.

O teste é o contrato. Pedido sem jeito de falhar vira opinião. Pedido com comando de teste vira trabalho que o agente, o cloud agent e o Bugbot conseguem repetir.

> **O ofício que sobra**
>
> Decidir o que construir, cortar o plano, ler o diff, recusar o comando estranho e escrever a regra no dia em que o erro repetir. O restante é ferramenta, e ferramenta você acabou de ver por nível.

AVANÇADO

Pratique o avançado

Cinco exercícios. Faça os três primeiros no seu repositório. Os dois últimos só se a conta tiver o recurso.

1. Crie `.cursor/agents/verificador.md` com `readonly: true` e a descrição de quando usar. Peça ao Agent para delegar a verificação de uma mudança pequena. Leia se ele realmente separou a checagem.
2. Peça `/create-hook` para um hook `beforeShellExecution` que apenas registra o comando, sem bloquear. Confira a saída na aba Hooks. Apague o hook se não for ficar.
3. Escreva três frases em `.cursor/permissions.json` no `block_instructions`: um comando que sempre pede aprovação no seu projeto. Teste com um pedido que tentaria esse comando.
4. Se a conta tiver Cloud Agent, mande uma tarefa com teste automático, feche o editor e leia o PR depois. Anote o que faltou no pedido. Essa nota vira a próxima linha do `AGENTS.md`.
5. Se o repositório estiver ligado ao Bugbot, abra um PR pequeno e leia os comentários. Marque o que era bug e o que era ruído. Ajuste o yaml só se o ruído for sistemático.

Quem fez o básico, o intermediário e estes exercícios já não depende de vídeo. O próximo capítulo é consulta, não aula.

AVANÇADO

Glossário e mapa rápido

Os nomes que o Cursor usa e os atalhos oficiais, num lugar só.

Glossário

As palavras que aparecem em inglês na tela, com o que fazer em cada uma, estão no básico: A tela está em inglês.

Nome	Significado neste livro
Tab	Sugestão enquanto você digita. Não obedece regra de projeto.
Edição inline	`Cmd+K` no editor. Mudança local. Sem regra de usuário.
Agent	Conversa que busca, edita e roda comando.
Ask	Modo só de leitura.
Plan	Pesquisa, pergunta, plano, e só então código.
Debug	Hipótese, log, você reproduz, correção curta, limpeza.
Design Mode	Apontar na tela, no navegador da janela de agentes.
Regra	Instrução persistente. Projeto, usuário, time ou `AGENTS.md`.
Skill	Procedimento sob demanda, chamado com `/` ou puxado pelo agente.
MCP	Ferramenta e dado de fora do repositório.
Subagente	Agente delegado, com contexto limpo.
Hook	Script no ciclo do agente.
Cloud Agent	Agente em máquina remota, com branch e pull request.
Bugbot	Revisão automática de pull request.
Auto-review	Modo de aprovação recomendado para comando, MCP e fetch.
Max Mode	Janela maior em planos antigos cobrados por request. Alguns modelos exigem.

Atalhos oficiais

Valores de outubro de 2026. Remapeie em Keyboard Shortcuts se precisar. No Windows e no Linux, use o botão do topo deste livro.

Geral

Atalho	Ação
`Cmd+I` e `Cmd+L`	Painel lateral
`Cmd+E`	Layout do agente

Atalho	Ação
`Cmd+.`	Menu de modos
`Cmd+/`	Percorrer modelos
`Cmd+Shift+J`	Configurações do Cursor
`Cmd+Shift+Space`	Voz
`Cmd+,`	Configurações gerais
`Cmd+Shift+P`	Paleta de comandos

Chat

Atalho	Ação
`Enter`	Enfileirar
`Cmd+Enter`	Enviar na hora, ou aceitar mudanças
`Cmd+Shift+Backspace`	Cancelar geração, ou rejeitar mudanças
`Cmd+Shift+L`	Seleção para o chat
`Shift+Tab`	Girar modos
`Cmd+N` ou `Cmd+R`	Conversa nova
`Cmd+T`	Nova aba
`Cmd+[` e `Cmd+]`	Conversa anterior e próxima
`Cmd+W`	Fechar conversa

Edição inline, Tab, terminal e Design

Atalho	Ação
`Cmd+K`	Abrir edição inline, ou a barra do terminal
`Option+Enter`	Pergunta rápida na edição inline. No chat, também fixa uma skill como modo.
`Tab` e `Cmd+→`	Aceitar sugestão, ou só a próxima palavra
`Cmd+Shift+D`	Ligar ou desligar o Design Mode

Fonte dos atalhos: cursor.com/docs/reference/keyboard-shortcuts. Fonte do restante: a documentação pública ligada em cada capítulo.